

Pawtrol

Privacy-first browser agent: build plan and 2-week roadmap
SIH Problem Statement 26171 (ISRO): On-Device Visual Perception for Lightweight Browser Agents

Updated: 17 Sep 2026 **Target:** working MVP by ~1 Oct 2026 **Status:** Day 1 complete

"We don't stop AI agents from seeing your browser. We make sure they only see what they're allowed to see, and sensitive data never has to leave your device at all."

1. Goal

Pawtrol is a Chrome extension that completes browser tasks on its own, quickly, while **never sending sensitive personal data** (passwords, Aadhaar, PAN, cards, names, faces) to cloud servers or LLMs. Sensitive values are swapped for tokens such as `<ID_1>` on the device. The cloud model reasons with the tokens, and the extension types the real values back in, locally.

SIH judging criterion	Weight	How we score it
How accurately the AI understands the screen	25%	On-device ONNX/WebGPU vision (text and face detection) combined with the page structure
How well it finds sensitive data	20%	Page structure, pattern matching with checksums (Verhoeff for Aadhaar, Luhn for cards), and OCR on pixels
How precisely it hides that data	20%	Token vault, pixel masking, and a re-OCR check that masked regions are unreadable
How light it runs on the device	20%	Quantized models, WebGPU, re-scanning only what changed; measured and reported
End-to-end speed	15%	Text-first context, batched actions, instant path for simple commands, per-stage timings

2. Where we are (Day 1 done)

- DONE** The extension was only minified build output. The core is now readable, tested TypeScript source.
- **Rebuilt source:** the service worker (agent loop, privacy pipeline, providers, learning), the content script, and the network tripwire, as about 25 modules in `src/`, built with esbuild.
 - **Proven equivalent** to the old extension: 25 automated tests run the new code side by side with the original on thousands of random inputs.
 - **Real-browser test:** Chromium runs a task on a page full of fake PII, against a mock AI model that records everything it receives. Result: **zero raw PII reached the model**, and the real Aadhaar number was filled in locally.
 - **Two bugs fixed** in the old code: digits were glued onto tokens (`7<ID_1>`), which corrupted what the AI saw, and tokens were lowercased, which broke them.
 - **Manually verified with Groq:** the model only ever saw `<ID_1>`, and the page's search box received `23456789 0124`.

What already works

- Token vault: sensitive values become tokens before egress and are resolved only on the device.
- Detection with checksums: Aadhaar (Verhoeff), cards (Luhn), PAN, IFSC, emails, phone numbers, API keys, names and addresses.
- Safety policy: refuses to type raw secrets and asks the user before irreversible clicks (pay, send, delete).
- Prompt-injection warning, network egress tripwire, hash-chained privacy ledger, and screenshot masking with an OCR check.
- Six AI providers (Anthropic, OpenAI, OpenRouter, Groq, NVIDIA, and local Ollama for zero egress).

3. Next build (Days 2–5): a real on-device vision pipeline

The gap we found: today's screenshot pipeline only masks what the page's code reports as sensitive, plus a crude skin-colour face finder. OCR only double-checks existing masks and never looks for new PII in the pixels. **An Aadhaar number drawn on a canvas, inside an image, in a PDF viewer, or on a scanned ID card is never detected.** The rules also require a real local vision model.

Goal: a real ONNX/WebGPU vision stage that looks at the rendered screen, finds text and faces in pixels, runs OCR on just those regions, applies the same PII detectors, masks what it finds, **fails closed**, and reports timings for each stage. It starts with pretrained models, so it works this week without waiting on training.

Team split (runs in parallel)

TEAMMATE Models and test pages

Python only; no extension code needed

1. **Download pretrained ONNX models** into `models/`, noting source, license and input size in `models/README.md`:
 - **PP-OCRv4 text detector** (PaddleOCR mobile, Apache-2.0, ~4.7 MB). Quantize to int8 with `onnxruntime.quantization`.
 - **YuNet face detector** (OpenCV model zoo, MIT, ~230 KB).
2. **Tricky test pages** in `bench/pages/`, each with a ground-truth `*.truth.json`:
 - Aadhaar/PAN on a `<canvas>` and inside a PNG
 - Fake ID card photo; page with faces
 - Password field with a misleading label
 - Zero-width characters inside numbers; text in an iframe
 - Prompt-injection page
3. **Stretch:** a Playwright crawler that auto-labels screenshots from page element boxes, for fine-tuning our own detector in Week 2.

EXTENSION Vision pipeline

TypeScript, in `src/offscreen/`

1. **Port the offscreen screenshot code** to TypeScript, verified against the original code as on Day 1.
2. **Vision stage** with `onnxruntime-web` (WebGPU, falling back to WASM): a text detector over the whole screen plus YuNet faces.
3. **Combine the signals:** text boxes the page's code doesn't cover (images, canvas, PDFs) get OCR'd, run through the Aadhaar/PAN/card detectors, and masked. Faces are always blurred.
4. **Fail closed:** if a model fails to load or times out, no screenshot is sent, and the failure is recorded in the ledger.
5. **Metrics:** per-stage timings (detection, OCR, masking, verification) plus the WebGPU/WASM backend.
6. **End-to-end test:** an Aadhaar number on a canvas and a face image. The screenshot the model receives must have no readable digits and a blurred face.

How the two tracks connect: the extension track starts immediately using the same public models. When the quantized models and benchmark pages land in `models/` and `bench/pages/`, they get plugged in and used for the benchmark.

4. Full 2-week roadmap

Days	Milestone	What gets built
Day 1 DONE	Source rebuild	TypeScript source, esbuild, side-by-side tests against the old code, real-browser E2E test, two privacy bugs fixed.
Days 2–5	On-device vision (<i>mandatory, 25%</i>)	Offscreen port; ONNX/WebGPU text and face detection; visual PII fusion; fail-closed screenshots; timings.
Days 5–6	Better detection	A small on-device name/address/organisation model (int8, under 30 MB); more Indian IDs: UPI, GSTIN, voter ID, driving licence, passport; fused confidence per finding.
Days 6–7	Fail-closed egress gate	A single choke point for everything sent to the cloud: a final re-scan, uncertain data blocked, and a text-only fallback if an image can't be verified.
Days 8–9	FastAPI gateway (server)	A server receives only sanitized data, runs a second independent PII scan (Presidio), rejects any leak, routes to the AI provider, and keeps an audit log of hashes only. Docker.
Days 9–10	Speed	Batched action plans, text-first context (images only on request), re-scanning only what changed, a small/large model split, replay of past successful tasks.
Days 10–11	Local secret fill and action verifier	An encrypted on-device profile (<code><PROFILE.pan></code>), checks before each action (element still present, label matches, token type fits the field, domain check).
Days 12–13	Benchmark and metrics	30–50 tricky pages; precision/recall, mask accuracy, leak count (target 0), latency, CPU/GPU/memory; a performance display in the side panel.
Day 14	Demo polish	Scripted demo: a government-style form, a "what the cloud saw" side-by-side view, an injection attempt, performance charts.

Architecture target

Chrome extension (on device)	FastAPI gateway (server)
• Perception: page structure + screenshot → ONNX/WebGPU vision → OCR on the detected regions • Sensitive-data detection combining page labels, OCR patterns, name/address detection and faces • Fail-closed egress gate (single choke point) • Token vault: real values never leave the device • Action verifier, then an executor that fills in real values locally	• Receives only a sanitized scene description • Second, independent PII scanner; rejects any leak • Routes to Gemini / Claude / OpenAI • Audit log (hashes only) and a metrics endpoint

5. Innovations that set us apart

1. **"What the cloud saw" replay.** Every step stores the exact sanitized data that was sent, shown next to the real page. It's proof, not a promise.
2. **Blind agent.** The AI completes whole forms without ever seeing a value. It works only with tokens, and those are type-checked locally.
3. **Prompt-injection firewall.** A token can only be typed on the site it came from, so a malicious page can't redirect your data elsewhere.
4. **Changed-region perception.** Vision re-runs only on parts of the screen that changed, with measured battery and CPU savings.
5. **Per-site privacy rules.** For example, banking sites send text only, with no images ever.
6. **Signed server receipts.** The gateway returns a signed "0 PII in N bytes" receipt, chained into a tamper-evident log.
7. **Fully offline mode.** A local Ollama model means zero egress, one toggle away.
8. **Skill replay.** Repeated tasks run from memory with zero AI calls: fast, and nothing can leak.

6. Priorities if time runs short

Must have	Cut first
Source rebuild DONE · ONNX/WebGPU vision · combined detection and name/address model · fail-closed gate · FastAPI gateway · local secret fill · benchmark numbers	Custom-trained face/ID detector (use pretrained instead) · skill replay · signed receipts

7. How we verify

- `npm run check` : type checks, unit tests (including side-by-side tests against the original code), build.
- `npm run e2e` : real Chromium plus a mock AI model; asserts zero raw PII in anything the model receives.
- **Manual check:** load `dist/` in Chrome, run a task on the PII test page, and inspect the request payload in DevTools. It should contain only tokens.
- **Benchmark (Week 2):** a report with precision/recall, mask accuracy, leak count, latency and resource use, for WebGPU and WASM.